

Certainly! Here's a complete Terraform command cheat sheet, from installation through deployment and cleanup.

Step 1: Verify Terraform Installation

</> Bash



```
terraform version
```

Example output:

Terraform v1.13.0



Step 2: Configure AWS Credentials

</> Bash



```
aws configure
```

Enter:

```
AWS Access Key ID: AKIA*****  
AWS Secret Access Key: *****  
Default region name: ap-south-1  
Default output format: json
```



Verify your credentials:

</> Bash



```
aws sts get-caller-identity
```

Step 3: Navigate to Your Terraform Project

Windows CMD:

</> cmd



```
cd C:\terraform-lab
```

Linux/macOS:

 Bash 

```
cd ~/terraform-lab
```

Check the files:

 Bash 

```
dir
```

or

 Bash 

```
ls
```

Expected files:

```
main.tf
variables.tf
outputs.tf
terraform.tfvars
```



Step 4: Initialize Terraform

 Bash 

```
terraform init
```

Reconfigure if needed:

 Bash 

```
terraform init -reconfigure
```

Upgrade providers:

 Bash 

```
terraform init -upgrade
```

Step 5: Format the Code

↵ Bash



```
terraform fmt
```

Format every Terraform file:

↵ Bash



```
terraform fmt -recursive
```

Check formatting only:

↵ Bash



```
terraform fmt -check
```

Step 6: Validate Configuration

↵ Bash



```
terraform validate
```

Step 7: Create an Execution Plan

↵ Bash



```
terraform plan
```

Save the plan:

↵ Bash



```
terraform plan -out=tfplan
```

Use a variable file:

↵ Bash



```
terraform plan -var-file="terraform.tfvars"
```

Step 8: Apply the Infrastructure

Interactive:

❏ Bash



```
terraform apply
```

Using the saved plan:

❏ Bash



```
terraform apply tfplan
```

Skip confirmation:

❏ Bash



```
terraform apply -auto-approve
```

Step 9: View Outputs

❏ Bash



```
terraform output
```

Specific output:

❏ Bash



```
terraform output public_ip
```

JSON output:

❏ Bash



```
terraform output -json
```

Step 10: Show Current State

 Bash



```
terraform show
```

Show state file:

 Bash



```
terraform show terraform.tfstate
```

Step 11: List Resources

 Bash



```
terraform state list
```

Example:

```
aws_vpc.main  
aws_subnet.public  
aws_instance.web  
aws_security_group.web
```



Step 12: Show a Specific Resource

 Bash



```
terraform state show aws_instance.web
```

Step 13: Refresh State

 Bash



```
terraform refresh
```

Note: In newer Terraform versions, a refresh is performed automatically during `plan` and `apply`.

Step 14: Destroy Resources

Preview destruction:

↗ Bash



```
terraform plan -destroy
```

Destroy interactively:

↗ Bash



```
terraform destroy
```

Destroy without confirmation:

↗ Bash



```
terraform destroy -auto-approve
```

Step 15: Workspace Commands

List workspaces:

↗ Bash



```
terraform workspace list
```

Create a workspace:

↗ Bash



```
terraform workspace new dev
```

Switch workspace:

↗ Bash



```
terraform workspace select dev
```

Show current workspace:

↗ Bash



```
terraform workspace show
```

Delete a workspace:

</> Bash



```
terraform workspace delete dev
```

Step 16: Provider Information

List providers:

</> Bash



```
terraform providers
```

Lock provider versions:

</> Bash



```
terraform providers lock
```

Step 17: Graph the Infrastructure

</> Bash



```
terraform graph
```

Step 18: Clean Up Local Files (if needed)

Delete Terraform working directory:

</> Bash



```
rm -rf .terraform
```

Delete state files (be careful):

</> Bash



```
rm terraform.tfstate*
```

Complete Workflow (Most Common)

 Bash 

```
terraform version
aws configure
aws sts get-caller-identity

terraform init
terraform fmt
terraform validate
terraform plan
terraform apply

terraform output
terraform state list
terraform show

terraform destroy
```

Common Troubleshooting Commands

If providers or state become inconsistent:

 Bash 

```
terraform init -upgrade
terraform init -reconfigure
terraform validate
terraform plan
terraform apply
```

These commands cover the complete lifecycle of a typical Terraform project on AWS.